

2.4. Transformaciones M2M

Las transformaciones modelo-a-modelo (M2M) toman un modelo de origen y producen un modelo destino, ambos conformes a sus respectivos metamodelos; dentro de MDA, el caso típico es el paso de un PIM a un PSM. El estándar del OMG que rige este tipo de transformación es QVT (Query/View/Transformation) v1.3 [1]. No obstante, la evidencia empírica reciente indica que la adopción industrial de lenguajes de transformación dedicados (MTL) como QVT o ATL sigue siendo limitada frente a lo que cabría esperar de un estándar consolidado: el estudio de Höppner et al. [2], basado en 56 entrevistas a investigadores y profesionales, documenta que la falta de tooling maduro y de evidencia sólida sobre sus ventajas frena su uso frente a lenguajes de propósito general.

Esta brecha entre el estándar y la práctica industrial explica un patrón frecuente en las herramientas MDD: muchas no exponen una etapa M2M diferenciada ni un lenguaje de transformación independiente conforme a QVT, sino que absorben esa lógica dentro del propio generador de código, traduciendo el modelo de origen directamente a texto sin materializar un PSM intermedio como artefacto propio. Esto no invalida la distinción conceptual entre M2M y M2T: evidencia, más bien, que en la práctica ambas etapas suelen fusionarse cuando el objetivo final es la generación de código y no la interoperabilidad entre modelos.

2.5. Transformaciones M2T

Las transformaciones modelo-a-texto (M2T) generan artefactos textuales código fuente, configuración o documentación a partir de un modelo conforme a un metamodelo; el estándar OMG de referencia es MOFM2T v1.0 [3]. Un ejemplo reciente de esta lógica se encuentra en el trabajo de Jiménez-Navajas et al. [4], quienes implementan transformaciones M2T con el lenguaje EGL (Epsilon Generation Language) para producir código Python y Qiskit a partir de modelos UML extendidos, lo que evidencia cómo un motor M2T recorre los elementos del modelo y los inyecta en plantillas parametrizadas para producir texto ejecutable.

Las herramientas MDD implementan esta lógica de dos formas distintas. Por un lado, motores declarativos como Acceleo o EGL exponen la plantilla como un archivo de texto editable que sigue, con distinto grado de fidelidad, la gramática propuesta por MOFM2T. Por otro lado, un grupo amplio de herramientas comerciales integra la generación como un módulo o plug-in compilado que lee las clases, atributos, relaciones y operaciones del modelo y las vuelca en código siguiendo plantillas fijas embebidas en el propio generador, sin exponer una gramática pública ni permitir su edición directa; esta segunda variante limita la personalización a los parámetros que la propia herramienta expone.

2.8. Plantilla / template

Una plantilla o template es el fragmento parametrizado que la herramienta MDD instancia por cada elemento del modelo para producir el código correspondiente. Este mecanismo puede implementarse como archivo de texto editable, ligado a una gramática de tipo MOFM2T, o puede empaquetarse como un componente compilado que aplica la definición del modelo sin exponerla como texto. Paolone et al. [5] documentan este segundo caso con xGenerator, una plataforma MDA propietaria que genera aplicaciones Java MVC completas a partir de un modelo UML sin exponer la plantilla al desarrollador, tratándola como una caja negra cuya salida solo puede evaluarse por la calidad del código resultante y no por inspección directa de la plantilla.

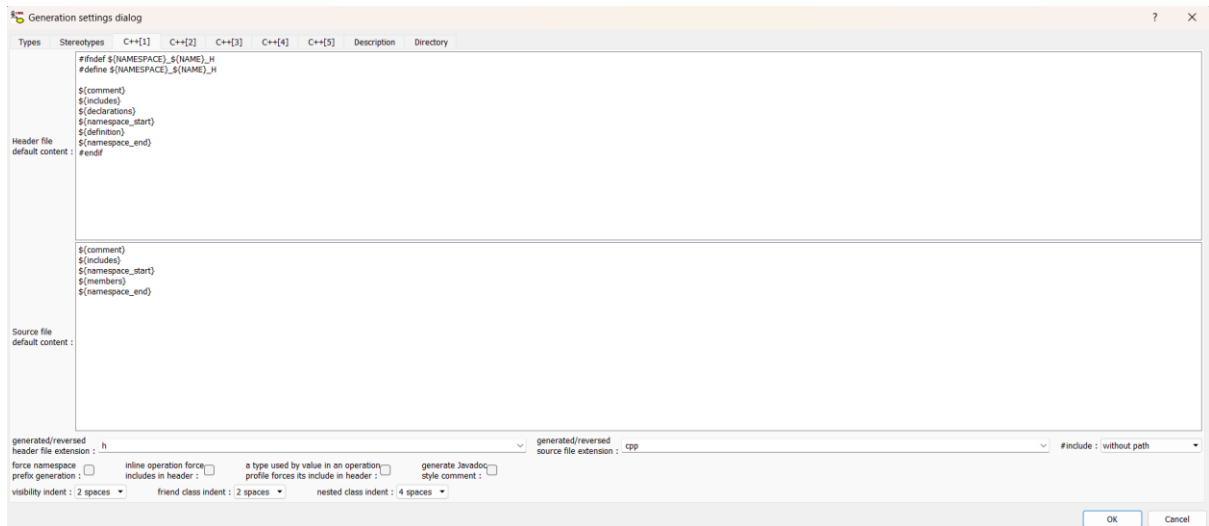
Un mecanismo de parametrización habitual en este tipo de generadores es el bloque protegido: toda operación cuyo cuerpo se delimita mediante marcadores reservados conserva, en regeneraciones sucesivas, el contenido que el desarrollador haya escrito manualmente, evitando que una nueva ejecución del generador sobrescriba la lógica de negocio ya implementada. Este mecanismo es coherente con la lógica M2T y con el estándar MOFM2T [3], y constituye la base técnica que permite un ciclo de roundtrip controlado: el modelo puede regenerarse repetidamente sin perder el trabajo manual, siempre que este se mantenga dentro de los delimitadores correspondientes.

5. Configuración del generador

5.1. Plantilla y parámetros

Para transformar el diagrama de clases validado en artefactos de implementación, se utilizó el generador C++ integrado de BOUML (versión 7.11 patch 4). En la pestaña Directory del cuadro de diálogo Generation settings, se estableció el directorio raíz de generación C++ (C++ root dir) apuntando a la carpeta generado/ dentro del proyecto. En la pestaña C++[1] se configuraron los parámetros base: extensión de cabecera h y de implementación cpp; directivas #include sin ruta; e indentación de 2 espacios para miembros y clases amigas, y 4 espacios para clases anidadas, junto con las guardas de inclusión automáticas (#ifndef / #define / #endif) y las plantillas de cabecera e implementación.

Adicionalmente, la pestaña C++[5] reveló que BOUML resuelve los tipos contenedores externos (vector, list, map, string) mediante una tabla de correspondencia que agrega automáticamente su directiva #include correspondiente junto con using namespace std;, por lo que el código generado referencia estos tipos sin el prefijo std:: explícito.



5.2. Política de regeneración y decisiones de mapeo

La pestaña C++[3] del generador confirmó las plantillas exactas que BOUML aplica según el tipo de relación: un atributo simple se declara como `${type} ${name}${value};`, mientras que las relaciones de asociación, agregación y composición con multiplicidad múltiple (*) se declaran anteponiendo un asterisco al nombre (`${type} * ${name}${value};`), es decir, mediante un puntero simple y no mediante un contenedor STL por defecto.

En consecuencia, la asociación Propietario (1) ↔ Mascota (0..*) se mapeó explícitamente a `vector<Mascota*> mascotas`; en `Propietario.h`, sustituyendo la plantilla nativa de puntero simple por el tipo contenedor vector definido en la pestaña C++[5], que ya incluye su `#include <vector>` y `using namespace std`; correspondientes.

Las operaciones de acceso (getters / setters) se generaron siguiendo la plantilla de la pestaña C++[4]: los métodos `get_${name}` se generan públicos, inline y const, mientras que los `set_${name}` se generan públicos sin modificadores adicionales, motivo por el cual el equipo ajustó manualmente la visibilidad y el calificador const de algunas operaciones críticas del dominio dentro de los bloques protegidos.

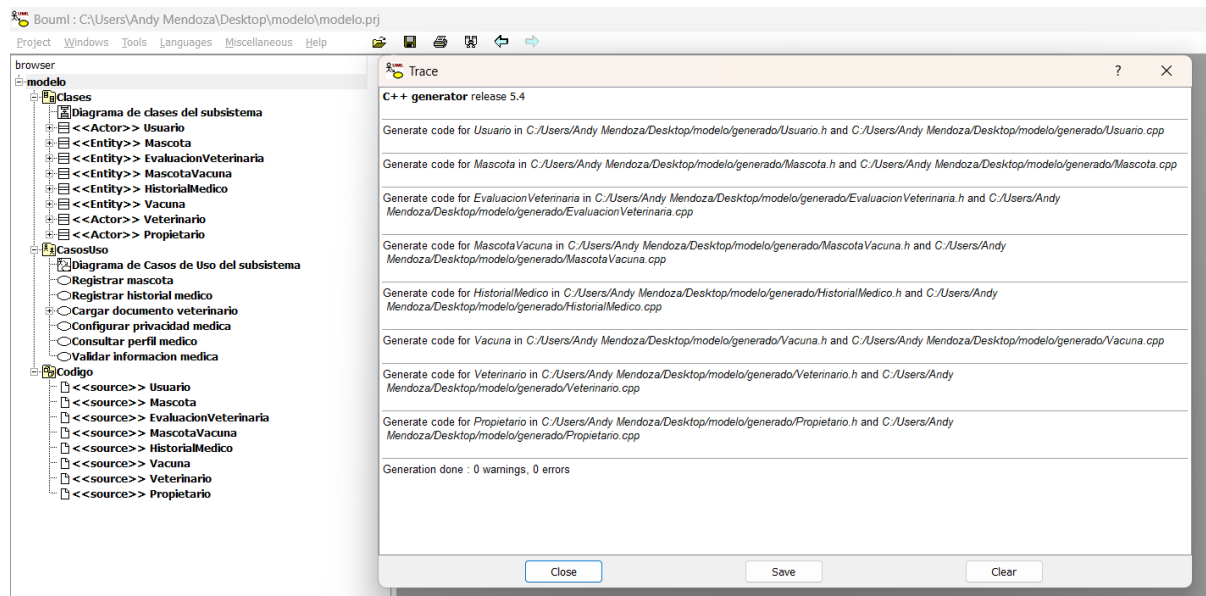
6. Proceso de generación

6.1. Ejecución y captura

La generación de código se ejecutó mediante la opción `Generate → C++` del menú contextual sobre el paquete raíz modelo, una vez confirmada la configuración descrita en la Sección 5.1. La consola de trazas de BOUML (Trace) reportó el uso del **C++ generator release 5.4** y confirmó el resultado `Generation done: 0 warnings, 0 errors`, evidenciando que las ocho

clases del modelo (Usuario, Propietario, Veterinario, Mascota, HistorialMedico, Vacuna, MascotaVacuna y EvaluacionVeterinaria) se procesaron correctamente.

Este proceso produjo dieciséis archivos —un par .h / .cpp por cada clase— sin errores ni advertencias, tal como se evidencia en la Figura 5. Dado el tamaño reducido del subsistema modelado, la generación se completó de forma prácticamente instantánea (menos de un segundo), sin que la consola de trazas reporte un cronómetro explícito para esta operación.

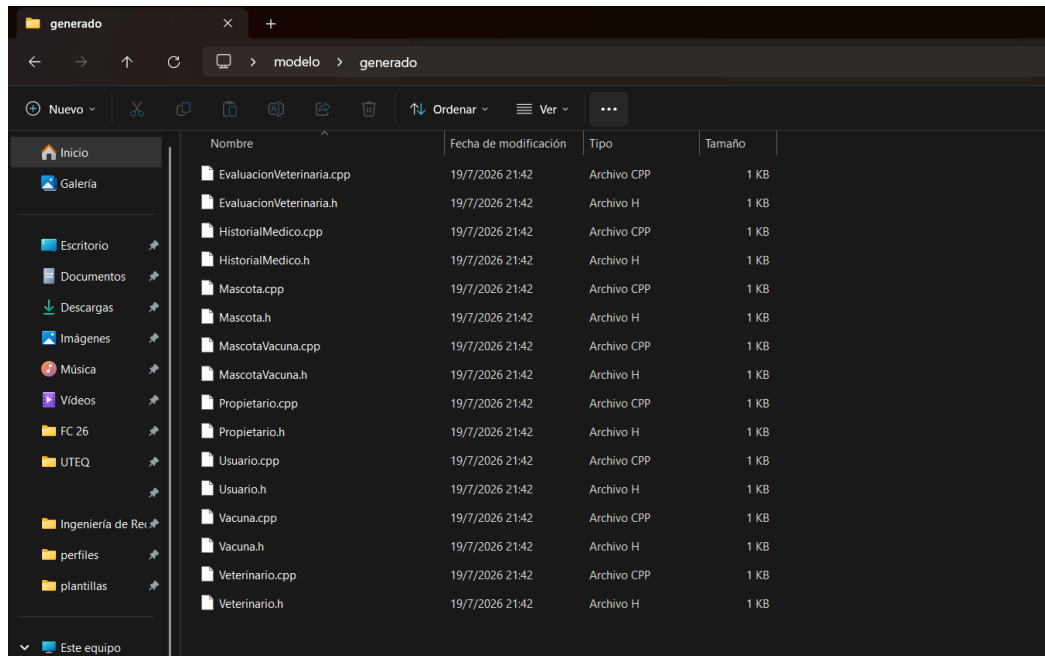


6.2. Árbol de proyecto y trazabilidad clase → archivo

Una vez confirmada la generación de forma exitosa, se verificó el contenido de la carpeta configurada como directorio raíz C++ (generado/), constatando la presencia de dieciséis archivos con una misma marca de tiempo, correspondientes a los ocho pares .h / .cpp de las clases del modelo. A continuación, se detalla el mapa de correspondencia física que documenta esta trazabilidad estructural:

- **Clase UML Usuario:** Archivos generados: Usuario.h / Usuario.cpp
- **Clase UML Propietario:** Archivos generados: Propietario.h / Propietario.cpp
- **Clase UML Veterinario:** Archivos generados: Veterinario.h / Veterinario.cpp
- **Clase UML Mascota:** Archivos generados: Mascota.h / Mascota.cpp
- **Clase UML HistorialMedico:** Archivos generados: HistorialMedico.h / HistorialMedico.cpp
- **Clase UML Vacuna:** Archivos generados: Vacuna.h / Vacuna.cpp

- **Clase UML MascotaVacuna:** Archivos generados: MascotaVacuna.h / MascotaVacuna.cpp
- **Clase UML EvaluacionVeterinaria:** Archivos generados: EvaluacionVeterinaria.h / EvaluacionVeterinaria.cpp



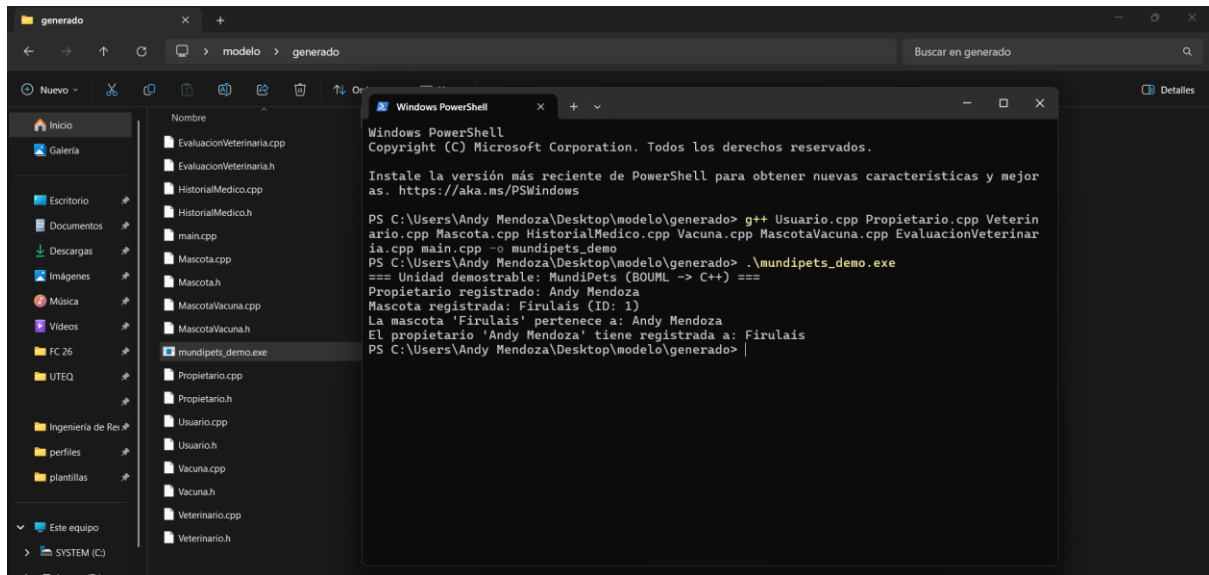
7. Verificación del código generado

7.1. Compilación y ejecución

El proyecto generado por BOUML se compiló utilizando el compilador **g++** (MinGW) mediante el siguiente comando, ejecutado sobre la carpeta **generado/**:

```
g++ Usuario.cpp Propietario.cpp Veterinario.cpp Mascota.cpp HistorialMedico.cpp
Vacuna.cpp MascotaVacuna.cpp EvaluacionVeterinaria.cpp main.cpp -o mundipets_demo
```

Como unidad mínima demostrable, se la elaboró un archivo `\texttt{main.cpp}` que instancia un objeto `Propietario` y un objeto `Mascota`, enlaza la asociación entre ambos mediante los punteros generados por BOUML (`mascota1.propietario` y `propietario1.mascotas`), y verifica en tiempo de ejecución la navegación bidireccional de dicha asociación. La compilación finalizó sin errores, generando el ejecutable `\texttt{mundipets_demo.exe}`, cuya salida en consola confirma que la relación entre ambas entidades quedó correctamente enlazada en memoria, tal como se muestra en la Figura 7.



7.2. Correspondencia modelo → código

A continuación, se detalla la trazabilidad entre los elementos definidos en el diagrama de clases de origen y los artefactos concretos que el generador C++ de BOUML produjo en la carpeta generado/:

- **Clase Mascota:** Mapeado al artefacto Mascota.h / Mascota.cpp.
- **Atributo idMascota (long, público):** Campo público long idMascota; en Mascota.h.
- **Atributo nombre (string, público):** Campo público string nombre; en Mascota.h.
- **Generalización Usuario → Propietario:** Estructurado como class Propietario : public Usuario en Propietario.h.
- **Asociación 1–N Propietario (1) ↔ Mascota (0..*):** Traducido en el puntero simple Propietario * propietario; en Mascota.h, y Mascota ** mascotas; en Propietario.h.
- **Composición Mascota ♦ HistorialMedico (1:1):** Instanciado como un miembro por valor HistorialMedico historial; en Mascota.h.
- **Asociación 1–N Mascota (1) ↔ EvaluacionVeterinaria (0..*):** Traducido en el contenedor vector<EvaluacionVeterinaria *> evaluaciones; en Mascota.h.
- **Asociación 1–N Veterinario (1) ↔ EvaluacionVeterinaria (0..*):** Traducido en el contenedor vector<EvaluacionVeterinaria *> evaluacionesRegistradas; en Veterinario.h.

- **Clase asociativa MascotaVacuna:** Mapeado a MascotaVacuna.h / MascotaVacuna.cpp, referenciada desde Vacuna.h mediante `vector<MascotaVacuna*>` aplicaciones;.

Esta correspondencia se validó de forma empírica en tiempo de ejecución, confirmando que la asociación generada refleja fielmente la relación definida en el modelo de origen.

7.3. Limitaciones detectadas

Durante el proceso de generación y verificación se identificaron cuatro limitaciones concretas del generador C++ de BOUML, todas corregidas manualmente antes de lograr una compilación exitosa:

- **Plantilla de asociación dependiente de un estereotipo vacío:** La plantilla por defecto para atributos derivados de asociaciones con multiplicidad múltiple incluía el marcador de estereotipo suelto cuando este campo venía vacío, dejando símbolos de apertura y cierre sueltos en el código generado y provocando errores de compilación en cascada, lo que requirió editar manualmente la plantilla global para retirar los símbolos sobrantes.
- **Asociaciones sin nombre de rol generadas como atributos anónimos:** Al editar el diagrama de clases, algunas asociaciones quedaron registradas sin un nombre de rol asignado en alguno de sus extremos, lo que BOUML tradujo como una declaración de atributo sin identificador, requiriendo su localización y remoción individual en el árbol del proyecto.
- **Ausencia de directivas de inclusión automáticas para tipos contenedor:** El tipo vector, utilizado en atributos derivados de asociaciones de multiplicidad múltiple, no generó automáticamente su directiva de inclusión correspondiente en todos los casos, requiriendo verificación manual para asegurar que cada archivo de cabecera que lo empleara incluyera el encabezado necesario.
- **Ausencia de métodos de acceso (getters y setters) generados por defecto:** Aunque la configuración del generador define plantillas para operaciones de tipo obtener y establecer un valor, BOUML no genera automáticamente estos métodos para los atributos del modelo. Ante esta limitación, el equipo optó por exponer como públicos los atributos estrictamente necesarios para la demostración, documentando esta decisión como una simplificación deliberada para efectos de la demostración en vivo.

Referencias

- [1] Object Management Group, “Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification,” Needham, MA, Jun. 2016. Accessed: Jul. 18, 2026. [Online]. Available: <https://www.omg.org/spec/QVT/1.3/PDF>
- [2] S. Höppner, Y. Haas, M. Tichy, and K. Juhnke, “Advantages and disadvantages of (dedicated) model transformation languages: A qualitative interview study,” *Empir. Softw. Eng.*, vol. 27, no. 6, Nov. 2022, doi: 10.1007/s10664-022-10194-7.
- [3] Object Management Group, “MOF Model to Text Transformation Language,” Needham, MA, Jan. 2008. Accessed: Jul. 18, 2026. [Online]. Available: <https://www.omg.org/spec/MOFM2T/1.0/PDF>
- [4] L. Jiménez-Navajas, R. Pérez-Castillo, and M. Piattini, “Code generation for classical-quantum software systems modeled in UML,” *Softw. Syst. Model.*, vol. 24, no. 3, pp. 795–821, Jun. 2025, doi: 10.1007/s10270-024-01259-w.
- [5] G. Paolone, R. Paesani, M. Marinelli, and P. Di Felice, “Empirical Assessment of the Quality of MVC Web Applications Returned by xGenerator,” 2021, doi: 10.3390/computers.